

ANFORDERUNGSANALYSE FÜR ENTWICKLER, TEIL 1

Strukturiert zerlegen

Slicing bietet klar definierte, systematische Schritte durch Anforderungen von außen nach innen, vom Groben zum Detail, vom breiten Scope zum dünn geschnittenen.

Die heute verfügbaren KI-Sprachmodelle machen klar, dass die Anforderungsanalyse in der Softwareentwicklung mehr Aufmerksamkeit benötigt. Ohne bessere Anforderungsanalyse kann das Potenzial der Sprachmodelle einfach nicht gehoben werden. Wenn Sie bisher noch kein Unwohlsein beim Umgang mit User Stories beschlissen hatte, sollten Sie es spätestens jetzt fühlen.

Mein fester Glaube ist, dass die Softwareentwicklung schon lange darunter leidet, dass sie sich zu früh auf Anforderungen einlässt. Ihre vorzeitige Optimierung bestand und besteht in vorzeitiger Entwicklung von Produktionscode. Doch bisher haben Menschen das einfach mit Ad-hoc-Interaktionen kompensiert. War dafür nicht die Co-Location im agilen Team Room gedacht?

KI hingegen braucht mehr Klarheit. Eine KI als Entwickler zu prompten, ohne selbst Anforderungen wirklich verstanden zu haben, scheitert ansonsten. Wer etwas von der KI will, bekommt beim Prompten den Stand des eigenen Verständnisses gespiegelt.

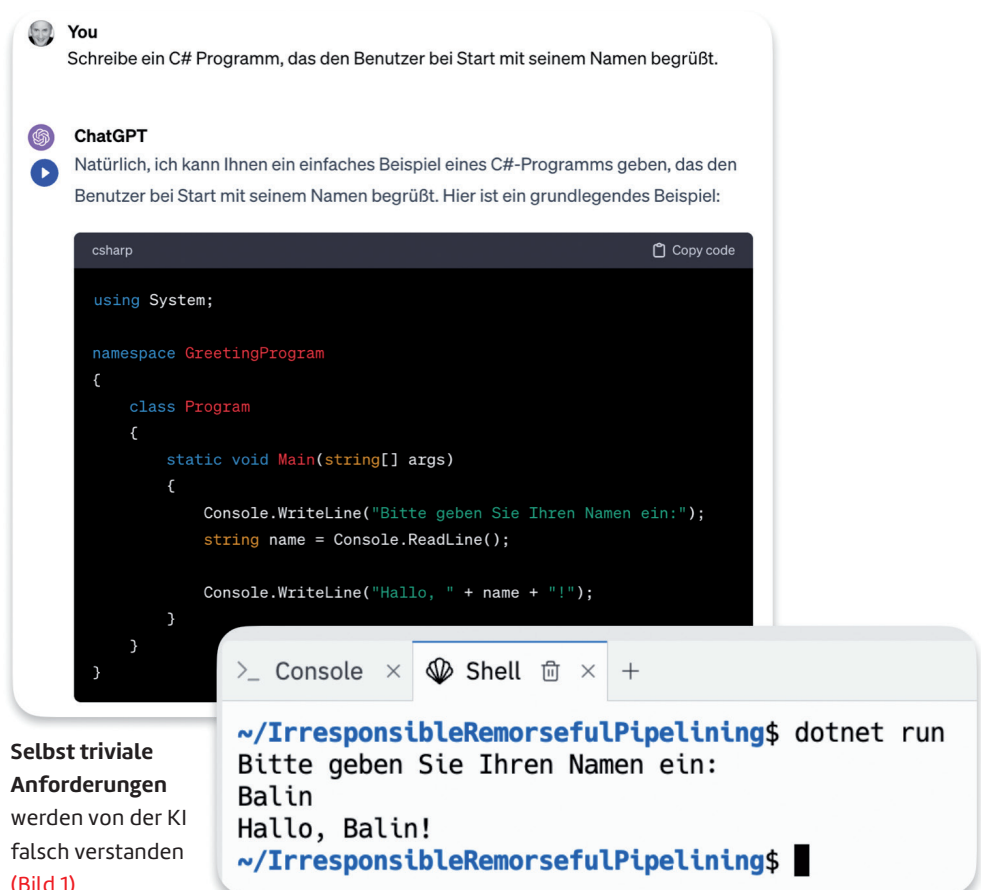
Dies mag das triviale Beispiel aus

Bild 1 verdeutlichen: Ich prompte ChatGPT, bekomme lauffähigen Code – und bin nicht zufrieden. Der Grund: Die KI hat einfach meine Anforderungen missverstanden.

Was war das Problem? Ich war nicht präzise genug. Die Begrüßung *Hallo, <Name>!* ist okay. Doch die Art, wie der Benutzer dem Programm seinen Namen mitgeben soll, entspricht nicht meinen Vorstellungen. Ich hatte mir ausgemalt, dass der Name gleich auf der Kommandozeile angegeben wird.

Aber so ist das eben: Was ich als Kunde im Kopf habe und nicht explizit ausdrücke, wird vom Entwickler – hier: ChatGPT – interpretiert. Die KI hat eine plausible Annahme getroffen.

Ich hätte mir nicht nur über meine Vorstellung klar sein, sondern diese auch noch ganz genau ausdrücken müssen. Im nächsten Anlauf, gezeigt in **Bild 2**, habe ich nachgebessert. Nun ist das Verhalten des Programms so, wie ich es mir gedacht hatte. Endlich. Oder jedenfalls fast, denn die KI hat wie-



der eine Lücke in meinen Anforderungen gefunden und durch eigene Entscheidung ausgefüllt: Ich hatte mir keine Gedanken darüber gemacht, was passieren soll, wenn kein Name als Parameter angegeben wird. Doch die KI fühlte sich unfähig, diesen Fall zu übergehen.

Als Entwickler haben Sie gelernt, dem Kunden oder dem Product Owner (PO) viele Fragen zu stellen. Ihnen wäre so etwas wie hier nicht passiert. Doch dieses Beispiel ist ja auch trivial. Bei größeren, realen, komplexeren Anforderungen tappt man dann aber doch leicht in dieselbe Falle wie ChatGPT: Man macht Annahmen – und weiß es womöglich nicht einmal. Unknown assumptions.

User Stories sollen „nur“ Anlass für Diskussionen sein. Klar, was denn auch sonst. Auf einen Post-it-Zettel passt nicht mehr als eine Anregung oder Erinnerung. Präzise Anforderungen sind viel, viel detailreicher. Doch wie sehen die aus?

Was sollte das persönliche Zielbild für Anforderungen sein? Wie kommt man da hin?

Meine Vorstellung von der Anforderungsanalyse ist, dass Entwickler den PO „ziehen“. Sie sitzen nicht brav da und hören sich an, was der PO blumig beschreibt. Nein, sie haben ein klares Bild davon, was das Ergebnis einer Anforderungsanalyse ist. Sie haben eine Vorstellung von der Form klarer Anforderungen. Sie legen den PO deshalb „auf den Grill“, bis ihr Ziel erreicht ist.

You
Schreibe ein C# Programm, das den Benutzer bei Start mit seinem Namen begrüßt. Der Benutzer gibt den Namen als Parameter auf der Kommandozeile mit.

ChatGPT
Um ein einfaches C#-Programm zu schreiben, das den Benutzer bei Start mit seinem Namen begrüßt, können Sie die Kommandozeilenargumente nutzen. Hier ist ein Beispielcode, der dies tut:

```
csharp
using System;

class Program
{
    static void Main(string[] args)
    {
        // Überprüft, ob ein Name als Kommandozeilenargument übergeben
        if (args.Length > 0)
        {
            Console.WriteLine("Hallo " + args[0] + "!");
        }
        else
        {
            Console.WriteLine("Hallo, bitte geben Sie Ihren Namen als A
        }
    }
}
```

```
>_ Console x Shell x +
~/IrresponsibleRemorsefulPipelining$ dotnet run
Hallo, bitte geben Sie Ihren Namen als Argument ein!
~/IrresponsibleRemorsefulPipelining$ dotnet run Dwalin
Hallo Dwalin!
~/IrresponsibleRemorsefulPipelining$
```

Um dieses Zielbild geht es mir in dem Ansatz, den ich Slicing nenne. In dem durchläuft die Anforderungsanalyse eine schrittweise Verfeinerung, die auf einen Punkt zuläuft. Dieser ist der Endpunkt der Analyse und der Startpunkt des Entwurfs. Slicing definiert für den Endpunkt eine Form; im Analyseprozess wird diese Form mit Inhalt aus der Domäne gefüllt, um die es dem PO geht.

Die Verfeinerungen des Slicing sind kompatibel mit dem agilen inkrementellen Vorgehen. Allerdings binden sie bei jedem Schritt den Schnitt durch die Anforderungen an Code an. Auf diese Weise wird die Anforderungsanalyse für Sie als Entwickler ein Handlauf zum Code.

Sie als Entwickler brauchen einen eigenen Blick auf Anforderungen. User Stories sind nicht genug. Ihr Verständnis soll sich in einer Weise entwickeln, dass Ihnen der anschließende Entwurf und die Codierung leichter fallen. Ihnen kann

es nicht egal sein, was Sie nach einem Anforderungsgespräch mit dem PO in den Händen halten. Und dem PO kann es eigentlich auch nicht egal sein, denn er möchte ja, dass Sie möglichst zügig seine Wünsche erfüllen.

Verständnis ist gut, Korrektheit ist besser

Wann hat man Anforderungen verstanden? Wie weiß der PO, dass man Anforderungen verstanden hat?

Die Antwort ist leider nicht: „Wenn ich dem PO erklären kann, was er haben will.“ Erklärungen sind schwach. Der PO will keine schönen Worte, sondern Taten. Worte, selbst wenn sie ihm plausibel erscheinen, sind mehrdeutig und lückenhaft. Seine wie Ihre. Anforderungen umzusetzen ist schlimmer, als „Stille Post“ zu spielen: Schon der Organisation ist letztlich unklar, wie Software ihr helfen kann und was sie sich also wünschen soll. Dem PO ist unklar, was die Organisation darüber weiß, was ihr helfen kann. Was der PO von seinem Verständnis ausdrückt, ist missverständlich, lückenhaft, gar fehlerhaft. Was Sie von dem, was der PO sagt, verstehen, ist unvollständig und fehlerhaft. Was Sie mit Ihrem Verständnis umsetzen, ist unvollständig und fehlerhaft.

Erst wenn Ihre Umsetzung am Ende den PO oder die Organisation erreicht, entsteht dort Klarheit, ob man schon gewusst hatte, was der Bedarf war, und ob Ihr Code ihn nun

deckt. Und wenn es schlecht läuft, hat der Bedarf sich seit dem initialen Wunsch sogar verschoben.

Das war und ist die Realität. Auch deshalb empfiehlt die Agilität, inkrementell vorzugehen und den Anforderungsball flach zu halten mit „Anlässen für Gespräche“. Sie setzt darauf, dass in Dialogen eher Missverständnisse, Lücken und Fehler aufgedeckt werden. Und sie setzt auf kleinere, schmalere Anforderungen, die Schritt für Schritt umgesetzt werden, um bei Missweisungen keine zu großen Nachbesserungen machen zu müssen.

Trotz solchen inkrementellen Vorgehens kommt allerdings auch die Agilität nicht um Iterationen herum. Selbst inner-

halb eines Inkrements werden Sie es schwer haben, in einem Durchgang von der Anforderung zu „perfektem“ Code zu kommen. Selbst in Bezug auf gegebene Anforderungen werden Sie Nachbesserungsrunden durchlaufen.

Das ist grundsätzlich nicht zu vermeiden, denke ich. Auch eine bessere Anforderungsanalyse schafft Iterationen nicht ab. Doch Iterationen sind kein Selbstzweck. Weniger ist besser. Auch leichtgewichtiger ist besser.

Wie können Sie da hinkommen? Alles beginnt damit, zu verstehen, was Verständnis ist beziehungsweise wie sich Verständnis „nachweisen“ lässt.

Sehen Sie sich die Anforderungen in Bild 3 an: Ich habe ChatGPT gebeten, Code zur Generierung von Zahlenfolgen zu schreiben. Diese Zahlenfolgen habe ich durch Beispiele beschrieben. ChatGPT hat mich verstanden. Woran erkenne ich das? Erkenne ich das Verständnis bei ChatGPT an dessen ►

**You**

Schreibe eine Funktion, die eine Zahlenreihe liefert. Die Zahlenreihe folgt einem Bildungsgesetz, einer Regel. Zurückgegeben werden sollen die ersten n Zahlen in der Reihe. Die Funktion dafür lautet in C# `f(int n)`.

Ich gebe dir jetzt Beispiele für Aufrufe mit verschiedenem n und zugehörigen Resultaten. Du lieferst mir dann die Funktion in C#:

- n : 1; Zahlenreihe: [1]
- n : 3; [1,2,3]
- n : 5; [1,2,3,4,5]

**ChatGPT**

Aus deinen Beispielen geht hervor, dass die Funktion eine Zahlenreihe von 1 bis `n` generieren soll. In C# kannst du dies erreichen, indem du ein Array der Größe `n` erstellst und es mit den entsprechenden Zahlen von 1 bis `n` füllst. Hier ist eine solche Funktion:

csharp

```
int[] f(int n)
{
    int[] sequence = new int[n];
    for(int i = 0; i < n; i++)
    {
        sequence[i] = i + 1;
    }
    return sequence;
}
```



Anforderungen für einfache
Zahlenfolgen werden von
ChatGPT korrekt verstanden
(Bild 3)

Erläuterungen, zum Beispiel „Die Zahlenreihe, die du beschreibst, folgt der Fibonacci-Sequenz, bei der jede Zahl die Summe der beiden vorhergehenden Zahlen ist, mit den Startwerten 1 und 1“? Das liegt nahe. Ich habe die Fibonacci-Sequenz im Kopf gehabt, und ChatGPT sagt, es habe aus den Beispielen herausgelesen, dass es sich um die Fibonacci-Zahlen handelt. Treffer!

Doch für mich ist nicht das wirklich Ausdruck von Verständnis. Oder selbst wenn, ist es recht unerheblich, denn in der obigen Kaskade der Anforderungsanalyse gibt es immer noch Schritt 5: die Umsetzung. Bei der Umsetzung kann auch das beste Verständnis noch stolpern.

Deutlicher wird das Verständnisprüfungsproblem in Bild 4. Eigentlich schlägt sich die KI ganz gut. Sie erkennt sofort, dass es sich um zwei Generierungsregeln handelt, die abwechselnd zum Einsatz kommen. Damit haben so manche Menschen ein Problem. Doch sollte ich damit zufrieden sein und glauben, dass ChatGPT wirklich verstanden hat?

Ich könnte jetzt genauer auf die Erläuterungen der KI schauen, um mir einen Eindruck von ihrem Verständnis zu verschaffen. Dabei würde mir auffallen, dass sie die zweite Regel nicht wirklich korrekt erkannt hat. Letztlich fällt mir

**You**

Hier ist eine andere Zahlenreihe. Ich gebe dir wieder Beispiele und du generierst die Funktion:

- n : 1; [1]
- n : 2; [1,1]
- n : 5; [1,1,2,3,5]

**ChatGPT**

Die Zahlenreihe, die du beschreibst, folgt der Fibonacci-Sequenz, bei der jede Zahl die Summe der beiden vorhergehenden Zahlen ist, mit den Startwerten 1 und 1. Hier ist eine C#-Funktion, die die ersten `n` Zahlen dieser Sequenz generiert:

csharp

Copy code

```
int[] f(int n)
{
    if (n <= 0) return new int[0]; // Keine Elemente, wenn n kleiner od
    if (n == 1) return new int[] { 1 }; // Nur ein Element in der Sequen

    int[] sequence = new int[n];
    sequence[0] = 1;
    sequence[1] = 1;

    for (int i = 2; i < n; i++)
    {
        sequence[i] = sequence[i - 1] + sequence[i - 2];
    }

    return sequence;
}
```

das aber nur auf, weil es sich wieder um ein sehr einfaches Beispiel handelt. Wäre es komplizierter, könnte ich bei der Erläuterung etwas übersehen oder hineininterpretieren.

Nein, die Erklärungen von ChatGPT sollten mir egal sein. Das, was zählt, ist ... der Code. „Im Code steckt die Wahrheit“, also lasse ich mir für den Code Testfälle aus meinen Beispielen generieren (Bild 5). Wenn ich die laufen lasse, zeigt sich ganz unmissverständlich: Irgendetwas ist faul.

ChatGPT hatte nicht verstanden oder sein Verständnis nicht zu hundert Prozent im Code abgebildet. Wo das Pro-

blem liegt, kann mir zunächst jedoch egal sein. Um festzustellen, dass es eine Lücke zwischen Wunsch und Wirklichkeit der Umsetzung gibt, muss ich nicht ChatGPT befragen, sondern den Code.

Den weiteren Dialog mit ChatGPT bis zum korrekten Code skizziert **Bild 6**. Es brauchte noch ein paar Anläufe, um die KI erkennen zu lassen, wie die Regel für die Zahlen mit geradem Index lautet. Bei keiner der Iterationen habe ich mich allerdings auf die Erklärungen der KI verlassen. Immer habe ich die neue Funktion sofort gegen die Tests laufen lassen.

In dieser Nusschale mit mir als Kunde und der KI als Entwickler zeigt sich für mich, was relevantes Verständnis der Anforderungen bedeutet: zufriedenstellendes Verhalten von Code. Ich fühle mich sogar versucht zu sagen: Verständnis ist sekundär. Verständnis allein hat keine Bedeutung. Verständnis ist nur ein Mittel, um Code herzustellen, der tut, was der Kunde von ihm erwartet.

Gerade wenn eine künstliche Intelligenz den Code generiert, liegt ja auch die philosophische Frage nahe: Kann eine KI überhaupt verstehen? Kann sie verstehen, wie wir meinen, dass Menschen verstehen?

Eine spannende Frage, finde ich. Lassen Sie uns darüber bei einem Bier plaudern. Für die Softwareentwicklung hingegen ist sie unbedeutend. Augenscheinlich kann KI codieren. Augenscheinlich kann sie Code schreiben, der unsere Formulierungen von Anforderungen mehr oder weniger erfüllt. Welches „Verständnis“ die KI von diesen Anforderungen hat, ist unerheblich, solange sie korrekten Code liefert. Die Korrektheit von Code allein ist der Lackmustest für die Umsetzung dessen, was ein PO an Wünschen geäußert hat.

Warum musste ich aber so viele Schleifen drehen, bevor die KI endlich die dritte Zahlenreihe korrekt erzeugen konnte (**Bild 6**)? Weil ich wieder sparsam war mit meinen Formulierungen. Ich habe für mich Offensichtliches nicht gesagt, sondern auf Intelligenz gesetzt, die schon verstehen wird, was ich meine. Hier künstliche Intelligenz, in anderen Fällen baut ein PO auf Ihre natürliche Intelligenz.

Mit etwas mehr Mühe bei der Anforderungsformulierung hätte es auch sofort geklappt, siehe **Bild 7**. Und noch immer nicht habe ich der KI die Regel für die Zahlenfolge mitgeteilt. Ob sie die aus den Beispielen ableiten konnte, zeigt allein die Überprüfung des Codes durch Tests.

Was tun, wenn der Code fehlschlägt, wie zum Beispiel in **Bild 5** zu sehen? Dann zeigt der Fehlerort an, welcher Teil der Anforderungen (!) nachzubessern ist. Hätte ich mich weniger dumm angestellt, hätte ich schon den zweiten Prompt auf die Zahlen mit geradem Index konzentriert.

Die Agilität hat im Grunde auch den Wert der Korrektheit als Beweis des Verständnisses erkannt und deshalb auf kurze Iterationszyklen gesetzt. Da sind zum einen die Sprints, die nach zwei Wochen oder sogar einer Woche Feedback liefern sollen. Zum anderen gibt es automatisierte Tests, die schon vorher die Reife einer Umsetzung beurteilen sollen.

Das geht in die richtige Richtung – allerdings bleibt eine Lücke in der agilen Anforderungsanalyse. Was ist denn das Ziel, wann kann sie als abgeschlossen betrachtet werden?

Testbarkeit für Korrektheit

Wenn sich ein wie immer geartetes Verständnis in der mehr oder weniger hohen Korrektheit von Code ausdrückt, dann kann das Ziel der Anforderungsanalyse nur sein, die Korrektheit auch leicht überprüfbar zu machen. ►



You

Und hier noch eine Zahlenreihe mit Beispielen:

```
- n:1; [10]
- n:2; [10,30]
- n:3; [10,30,11]
- n:4; [10,30,11,28]
- n:5; [10,30,11,28,12]
- n:6; [10,30,11,28,12,24]
- n:8; [10,30,11,28,12,24,13,16]
```



ChatGPT

Die von dir beschriebene Zahlenreihe scheint zwei unterschiedliche Muster zu befolgen, abhängig davon, ob der Index der Zahl gerade oder ungerade ist. Bei ungeraden Indizes steigt die Zahl um 1 an, beginnend mit 10. Bei geraden Indizes fällt die Zahl um 2 ab, beginnend mit 30. Diese beiden Muster wechseln sich ab.

Die C#-Funktion, um diese Zahlenreihe zu generieren, könnte so aussehen:

```
csharp Copy code

int[] f(int n)
{
    int[] sequence = new int[n];
    int oddNumber = 10; // Startwert für ungerade Indizes
    int evenNumber = 30; // Startwert für gerade Indizes

    for (int i = 0; i < n; i++)
    {
        if (i % 2 == 0) // Überprüfe, ob der Index gerade ist
        {
            sequence[i] = oddNumber;
            oddNumber++; // Erhöhe die Zahl für den nächsten ungeraden
        }
        else // Für ungerade Indizes
        {
            sequence[i] = evenNumber;
            evenNumber -= 2; // Verringere die Zahl für den nächsten ge
        }
    }

    return sequence;
}
```

Anforderungen für kompliziertere Zahlenfolgen werden nicht verstanden (**Bild 4**)

Schon die Anforderungsanalyse läuft also auf Testbarkeit hinaus. Ihr Verständnis ist sekundär, die Testbarkeit Ihres Codes ist primär. Wenn er testbar ist, muss der PO Sie nicht fragen: „Haben Sie das verstanden?“, sondern er lässt einfach Tests für die Qualität Ihres Codes sprechen. Mein Dialog oben mit der KI zeigt das beispielhaft.

Was aber braucht es für Testbarkeit? Ansatzpunkte. Ihr Code muss von vornherein Ansatzpunkte für automatisierte Tests haben, die dem PO Auskunft über seine Reife geben (also den Grad der schon erreichten Korrektheit, die Anforderungskonformität).

Solche Ansatzpunkte sind... Funktionen.

Im obigen Fall der Zahlenreihengenerierung war die Funktion *f()* der natürliche Ansatzpunkt. Als Kunde hatte ich mir schon eine Funktion gewünscht. Doch das ist nicht der Normalfall. Sie können von einem PO nicht erwarten, dass er Ihnen sagt: „Schreiben Sie eine Funktion *f()*, die dieses und jenes tut.“

Diese Präzision ist jedoch nötig, damit man Testbarkeit bekommt.

Besonders deutlich wird das, sobald Sie eine KI für die Codegenerierung zu Hilfe nehmen. Wie wollen Sie sonst ihr Elaborat überprüfen? Sie sind ihr gegenüber der Kunde, der auf einen Blick sehen will, ob sein Wunsch schon erfüllt ist.

Das wird mittelfristig auch so bleiben, denke ich. Nicht der PO wird schon morgen einer KI sagen, welche Software sie „ausspucken“ soll. Nein, Sie werden als Mittelsmann zwischen PO und KI stehen. Sie werden derjenige sein, der fremdem Code vertrauen muss. Bisher war das der PO; er konnte sich nur Ihrer technischen Expertise als Entwickler anheimgeben. Nun ist es an Ihnen, jemand anderem zu vertrauen: der KI. Fragen Sie sich selbst: Was brauchen Sie dafür?

Die Antwort: Funktionen, die automatisiert getestet werden können.

Solche Funktionen aus Anforderungen abzuleiten, das ist für mich der Zweck von Slicing (vergleiche Bild 8).

Wie hätte das bei der ersten Aufgabe ausgesehen? Dort hat nur ein Programmablauf, also ein manueller Test, die Inkorrektheit aufdecken können. Wie anders hätte der Prompt, also die Anforderungsformulierung ausgesehen, wäre das Ideal schon eine Funktion gewesen, für die Code hätte erzeugt werden sollen?

Wäre eine Funktion *begrüßen()* passend gewesen? Nein, denn wie soll sie getestet werden? Zwar sind hinter ihrer Si-

You

Generiere mir ein Programm, das diese letzte Funktion in mehreren `Debug.Assert()` mit den Testfällen aufruft.

ChatGPT

Um die Funktion `f` mit verschiedenen Testfällen zu prüfen, können wir das Programm mit `Debug.Assert()`-Aufrufen versehen. Diese Aufrufe überprüfen, ob das Ergebnis der Funktion `f` mit den erwarteten Werten übereinstimmt. Hier ist ein Beispielprogramm in C#, das dies tut:

```
csharp
using System;
using System.Diagnostics;

class Program
{
    static void Main()
    {
        TestFunction();
        Console.WriteLine("Alle Tests bestanden.");
    }

    static void TestFunction()
    {
        Debug.Assert(AreArraysEqual(f(1), new int[] { 10 }), "Testfall 1");
        Debug.Assert(AreArraysEqual(f(2), new int[] { 10, 30 }), "Testfall 2");
        Debug.Assert(AreArraysEqual(f(3), new int[] { 10, 30, 11 }), "Testfall 3");
        Debug.Assert(AreArraysEqual(f(4), new int[] { 10, 30, 11, 28 }), "Testfall 4");
        Debug.Assert(AreArraysEqual(f(5), new int[] { 10, 30, 11, 28, 1 }), "Testfall 5");
        Debug.Assert(AreArraysEqual(f(6), new int[] { 10, 30, 11, 28, 1, 1 }), "Testfall 6");
        Debug.Assert(AreArraysEqual(f(8), new int[] { 10, 30, 11, 28, 1, 1, 1, 1 }), "Testfall 8");
    }

    static bool AreArraysEqual(int[] arr1, int[] arr2)
    {
        if (arr1.Length != arr2.Length)
        {
            return false;
        }
        for (int i = 0; i < arr1.Length; i++)
        {
            if (arr1[i] != arr2[i])
            {
                return false;
            }
        }
        return true;
    }
}
```

Testfälle für die Implementation der dritten Zahlenreihe (Bild 5)

```
> Console x Shell x +
~/IrresponsibleRemorsefulPipelining$ dotnet run
Process terminated. Assertion failed.
Testfall für n=6 fehlgeschlagen
   at Program.TestFunction() in /home/runner/IrresponsibleRemorsefulPipelining/main.cs:line 19
   at Program.Main() in /home/runner/IrresponsibleRemorsefulPipelining/main.cs:line 8
~/IrresponsibleRemorsefulPipelini
```

gnatur alle Details verborgen – die Beschaffung des Namens wie die Ausgabe der Begrüßung –, doch genau daraus ergibt sich schlechte Testbarkeit. Seiteneffekte wie Zugriff auf die Konsole lassen sich schwer automatisiert überprüfen.

Spüren Sie, wie die Testbarkeit an der Analyse „zieht“? Als Erstes kommen Sie darauf, den Scope der Funktion zu verengen: *string begrüßungFormulieren(string name)*. In der Funktion steckt immer noch testwürdige Logik, doch es wurde einige Logik „herausgepresst“. Die kann in zwei weitere Funktionen verpackt werden:

```
string nameBeschaffen()
begrüßen(string begrüßung)
```

Diese beiden Funktionen enthalten weiterhin die Seiteneffekte und lassen sich schwer testen. Viel wichtiger ist jedoch zunächst, dass die Frage explizit im Raum steht: Woher kommt der Name und wohin geht die Begrüßung?

Solange der Name von der Konsole eingelesen werden soll, ist *nameBeschaffen()* besonders widerspenstig beim ►

 You

Dein Verständnis ist immer noch inkorrekt. Die neue Funktion schlägt wie folgt fehl:

```
###
Testfall für n=4 fehlgeschlagen
at Program.TestFunction() in
/home/runner/IrresponsibleRemorsefulPipelining/main.cs:line 17
at Program.Main() in /home/runner/IrresponsibleRemorsefulPipelining/main.cs:line 8
###
```

Schau nochmal genau hin. Analysiere Schritt für Schritt. Es ist sehr wichtig, dass du eine Funktion mit korrektem Code generierst, der die Regeln der Zahlenreihe implementiert.

 You

Das ist immer noch nicht korrekt. Schau dir die Entwicklung der Zahlen in der zweiten Zahlenreihe (Index 2, 4, 6 usw.) nochmal an. Denke Schritt für Schritt.

 You

Das ist schon besser. Aber schau noch genauer hin und denke Schritt für Schritt für die zweite Regel. Schau alle Beispiele an und generiere am Ende den Code.

```
n: 1; [10]
n: 2; [10, 30]
n: 3; [10, 30, 11]
n: 4; [10, 30, 11, 28]
n: 5; [10, 30, 11, 28, 12]
n: 6; [10, 30, 11, 28, 12, 24]
n: 8; [10, 30, 11, 28, 12, 24, 13, 16]
```

 You

Du hast die Differenzen der Werte der Zahlen mit den Indizes 2,4,6 usw. korrekt berechnet. Doch du hast den falschen Schluss daraus gezogen. Die Regel für ihre Entwicklung lautet anders. Schau dir die Differenzen nochmal genau an.

Hier nochmal die Beispiele:

```
n: 1; [10]
n: 2; [10, 30]
n: 3; [10, 30, 11]
n: 4; [10, 30, 11, 28]
n: 5; [10, 30, 11, 28, 12]
n: 6; [10, 30, 11, 28, 12, 24]
n: 8; [10, 30, 11, 28, 12, 24, 13, 16]
```

 You

Hier sind die Zahlen der zweiten Reihe: 30, 28, 24, 16

Leite daraus die Bildungsregel ab. Denke Schritt für Schritt.

```
csharp Copy code

int[] f(int n)
{
    int[] sequence = new int[n];
    int oddIndexNumber = 10; // Startwert für ungerade Indizes
    int evenIndexNumber = 30; // Startwert für gerade Indizes
    int decreaseStep = 2; // Anfangswert der Differenz für gerade Indizes

    for (int i = 0; i < n; i++)
    {
        if ((i + 1) % 2 == 1) // Ungerade Indizes (1-basiert)
        {
            sequence[i] = oddIndexNumber;
            oddIndexNumber++; // Erhöhe die Zahl für den nächsten ungeraden Index
        }
        else // Gerade Indizes
        {
            sequence[i] = evenIndexNumber;
            evenIndexNumber -= decreaseStep; // Verringere die Zahl für den nächsten geraden Index
            decreaseStep *= 2; // Verdopple die Differenz für den nächsten geraden Index
        }
    }

    return sequence;
}
```

Es braucht mehrere Anläufe, um die KI die korrekte Bildungsregel für die Zahlenreihe „verstehen“ zu lassen (Bild 6)

**You**

Schreibe eine Funktion, die eine Zahlenreihe liefert. Die Zahlenreihe folgt einem Bildungsgesetz, einer Regel. Zurückgegeben werden sollen die ersten n Zahlen in der Reihe. Die Funktion dafür lautet in C# `int[] f(int n)`.

Ich gebe dir jetzt Beispiele für Aufrufe mit verschiedenem n und zugehörigen Resultaten. Du lieferst mir dann die Funktion in C#:

```
...
n: 1; [10]
n: 2; [10, 30]
n: 3; [10, 30, 11]
n: 4; [10, 30, 11, 28]
n: 5; [10, 30, 11, 28, 12]
n: 6; [10, 30, 11, 28, 12, 24]
n: 8; [10, 30, 11, 28, 12, 24, 13, 16]
...
```

Beachte, dass diese Zahlenreihe durch zwei unterschiedliche Regeln für Zahlen bei ungeradem und geradem Index gebildet wird. Für jede Regel schaue dir nur die entsprechenden Zahlen und ihre Differenzen an und leite aus ihren Differenzen die Entwicklung ab. Berücksichtige alle Potentiale. Denke Schritt für Schritt.

Präzisere Anforderungen führen aus dem Stand zu einem besseren Ergebnis (Bild 7)

csharp

Copy code

```
int[] f(int n)
{
    if (n <= 0) return new int[0];

    int[] sequence = new int[n];
    int oddNumber = 10;
    int evenNumber = 30;
    int evenDiff = 2;

    for (int i = 0; i < n; i++)
    {
        if (i % 2 == 0) // Ungerade Positionen (beginnend bei 0)
        {
            sequence[i] = oddNumber;
            oddNumber++;
        }
        else // Gerade Positionen
        {
            sequence[i] = evenNumber;
            evenNumber -= evenDiff;
            evenDiff *= 2; // Die Differenz verdoppelt sich bei jedem
        }
    }

    return sequence;
}
```

Testen. Das triggert bei Ihnen hoffentlich die Frage: Muss der Name wirklich von der Konsole eingelesen werden? Hat der PO das so formuliert? Was wäre einfacher zu testen?

Eine gut testbare Funktion als Ziel der Anforderungsanalyse führt somit ... ins Gespräch – und zwar in ein sehr konkretes, zielgerichtetes. Ihr Ideal Testbarkeit lässt Sie am PO „ziehen“. Sie können die Analyse geführt durch ein Strukturelement des Codes mit Fragen vorantreiben: „Verstehe ich das richtig, dass das Programm nach dem Namen fragen soll?“, „Die Umsetzung ginge schneller und wäre besser zu überprüfen, wenn der Name von der Kommandozeile gelesen würde. Wäre das nicht auch eine Option? Das ließe sich einfacher umsetzen und testen“, und so weiter.

Wie ein Trüffelschwein sollten Sie also während der Anforderungsanalyse auf der Suche nach testbaren Funktionen sein. Sie stecken überall in den User Stories (oder wie auch immer Ihnen Anforderungen präsentiert werden). Mal ist es eine Funktion für ein Post-it, mal mehrere.

In jedem Fall halte ich die Anforderungsanalyse für unvollendet, solange nicht klar ist, welche Funktionen hinter den Wünschen stecken:

- Welche Funktionen werden gebraucht?
- Wie sieht ihre Signatur aus?

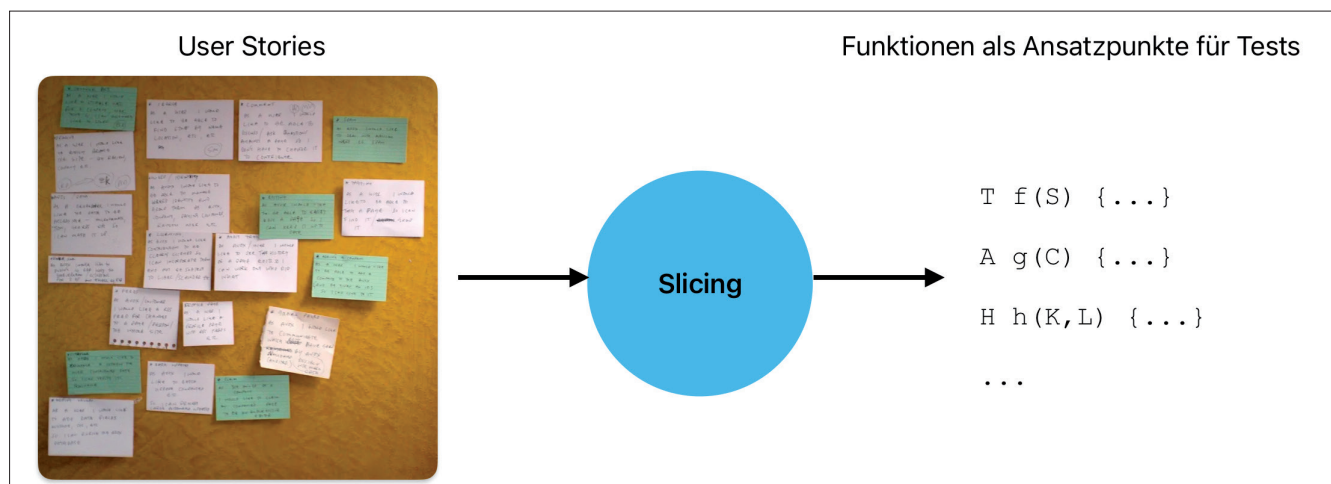
gen in Testfällen formalisiert wurden. Rein natürlichsprachliche Anforderungen sind nicht genug!

Im ersten Beispiel waren die Anforderungen nur natürlichsprachlich. Das schlechte Ergebnis war also kein Wunder. Im zweiten Beispiel waren sie natürlichsprachlich und formal präzise, wenn auch nicht codiert. Die Beispiele waren für ChatGPT genug, um daraus erstens Produktionscode abzuleiten und zweitens Tests zu formulieren.

„Erklären ist gut, Tests sind besser“, könnte ich frei nach Lenin formulieren. Ja, erklären Sie dem PO gern, was Sie nach seiner Präsentation der Anforderungen verstanden haben. Damit reflektieren Sie aktiv und vertiefen Ihr Verständnis. Gleichzeitig hat der PO eine Chance, darin Lücken, Missverständnisse, Fehler zu erkennen – und auch er wird dadurch angeregt, über seine Vorstellung nochmals zu reflektieren. Super!

Doch Sie beide sollten eine solche Erklärung nicht überbewerten. Sie ist nicht mehr als vorläufig und ein Surrogat. Sie ist nicht das Real Thing. Denn das ist nur korrekter Produktionscode.

Sich in Analysedialogen zu drehen hat das primäre Ziel, Klarheit in Form von Testbarkeit herzustellen. Es geht um das Tupel (*Funktionssignatur, Testfälle*).



Slicing leitet aus Anforderungen testbare Funktionen ab (Bild 8)

- Gibt es Seiteneffekte? Wenn ja, wie können die „herausgedrückt“ werden, um die Testbarkeit zu erhöhen?
- Und schließlich: Welche Tests werden für eine belastbare Aussage über die Korrektheit gebraucht?

Indem Ihr Sinnen und Trachten während der Analyse auf Funktionen als Test-Ansatzpunkte ausgerichtet ist, führen im Grunde Sie den PO. Nicht er sagt an und Sie folgen, sondern Sie haben einen (unausgesprochenen) Anspruch, den er erfüllen soll.

Ihr Anspruch ist sogar doppelt, wenn Sie den PO nicht gehen lassen, ohne klare Testfälle mit ihm abgesprochen zu haben. Die Entwicklung des Codes hinter der Fassade der Ansatzpunkte sollte nicht beginnen, ohne dass die Anforderun-

Was darüber hinaus vermittelt wird, steht unter hohem Risiko, unknown Unknowns auszulassen. Darauf müssen alle Seiten vorbereitet sein.

Und was ist es, das es mehr braucht als (*Funktionssignatur, Testfälle*)? Die Beschreibung einer „Kausalkette“ (Bild 9): Wie führt der Input für die Funktion zusammen mit Zustand, auf den sie zugreifen kann, zu Output und neuem Zustand?

Diese „Kausalkette“ jedoch ist oft nicht klar. Oder anders gesagt: Sie zu erarbeiten ist geradezu Ihre Aufgabe als Entwickler. Deshalb ist es umso wichtiger, zumindest eine Vorstellung von Testfällen zu haben. Der PO kann nicht sagen: „Hier ist eine ‚Kausalkette‘, aber ich habe keine Ahnung, wie Testfälle dafür aussehen.“ Wenn er ein Verständnis von der „Kausalkette“ hat, muss er auch verstehen, wie ihre ►

Wirkung ist. Allerdings kann er sagen „Hier sind Testfälle. So sehen Ergebnisse für gewisse Eingaben aus. Aber ich habe keine Ahnung, wie dazu die ‚Kausalkette‘ aussieht.“

Üblicherweise konzentriert sich die Anforderungsdiskussion auf die „Kausalkette“, hinter der ein bestimmtes zu lösendes Problem steht. Ich denke jedoch, dass sich die Diskussion hin zu Funktionen und Testfällen verschieben sollte.

Prototypen gegen Unklarheit

Weniger Klarheit finde ich nicht akzeptabel für Sie als Entwickler. Der PO soll liefern, wenn er etwas haben will. Er soll sich auf den Hosenboden setzen und sich selbst erst mal klar darüber sein, was er will. Das beweist er, wenn er fähig ist, Testfälle zu formulieren. Sie helfen ihm gern, diese Testfälle zu formalisieren und Funktionen zu finden, auf die sie ange-setzt werden können. Anders ist die „Reifeprüfung“ (und auch die Stabilitätskontrolle) von Produktionscode nicht skaliertbar möglich.

Aber was, wenn die Anforderungen sich einfach noch nicht so klar formulieren lassen? Was, wenn sich Ihnen noch keine Funktion anbietet, hinter deren Signatur die Anforderungen implementiert werden könnten, um durch sie mit automatisierten Tests überprüft zu werden? Was, wenn weder dem PO noch Ihnen (gute) Testfälle einfallen?

Der Grund dafür kann sein, dass der Scope der Anforderungen zu groß ist. Dann gilt es, feiner und feiner zu schneiden, bis der Scope so überschaubar ist, dass sich das Tupel formulieren lässt.

Es kann allerdings auch sein, dass der PO noch nicht genau weiß, was er wirklich will. Er hat noch keine exakte Vorstellung davon, wie korrekte Transformationen aussehen. Die erste Aufgabe (Bild 1) könnte dafür ein Beispiel sein. Auf die Frage „Wie soll der Benutzer dem Programm denn den Namen mitteilen?“ könnte der PO antworten: „Hm ... keine Ahnung. Auf der Kommandozeile ist es einfach, aber vielleicht unbequem. Gefragt zu werden ist schlechter zu testen, aber bequemer. Ich kann mich noch nicht entscheiden ...“

Was tun in einem Fall von Unklarheit, Unsicherheit, Entscheidungsmangel? Für mich ist die Antwort: Prototyping.

Prototypen können alle möglichen Formen haben; sie müssen nicht codiert sein. Für Benutzerschnittstellen könnten Sie zum Beispiel auch Papier benutzen.

Prototypen simulieren Systemverhalten. Produktionscode wird für sie nicht geschrieben oder verändert. Deshalb simulieren sie nur. Sie sind nicht das Produktionssystem, sondern tun nur so. Sie vermitteln eine für die auszumerkende Unklarheit relevante Anmutung, wie das Produktionssystem aussehen beziehungsweise sich verhalten könnte.

Für das erste Beispiel könnten Sie zum Beispiel zwei Prototypen mit einem Shell-Skript schreiben (Bild 10). Das ist in diesem trivialen Beispiel nicht viel weniger Aufwand als der C#-Code. Doch die andere Form vermittelt Ihnen hoffentlich schon ein Gefühl dafür, was ich damit meine, wenn ich sage, Prototypen seien kein Produktionscode.

Sie bestehen nur aus dem Allernötigsten, um den PO in die Lage zu versetzen, eine Entscheidung zu treffen, die ihn formaler Klarheit näher bringt, die sich für Sie in Funktionssignaturen und Testfällen ausdrückt.

Solange Ihre Ansprüche an die Testbarkeit von Umsetzungen der Anforderungen nicht erfüllt sind, lassen Sie besser die Hände von Produktionscode! Seien Sie natürlich dem PO stets behilflich beim Ausräumen von Unsicherheiten. Produktionscode ist dafür jedoch kein Mittel. Prototypen sind es.

In Zeiten von KI als „Codierknecht“ verschiebt sich Ihre Rolle immer weiter in Richtung Geburtshelfer für Anforderungsklarheit. Die Codierung übernimmt später die KI. Sie kann jedoch (noch) nicht so hilfreich beim Herauskitzeln von Klarheit sein. Dafür braucht es mehr Interaktion, mehr eigenständige Kreativität und Kontextkenntnisse. Hier ist der Ansatzpunkt, an dem Sie mit natürlicher Intelligenz und Einfühlungsvermögen glänzen können.

Eine klare Formulierung der Anforderungen ist jedoch nicht zu verwechseln mit Verständnis. Im Zweifelsfall verstehen Sie die Anforderungen nicht – erkennen jedoch deren mehr oder weniger formale Klarheit.

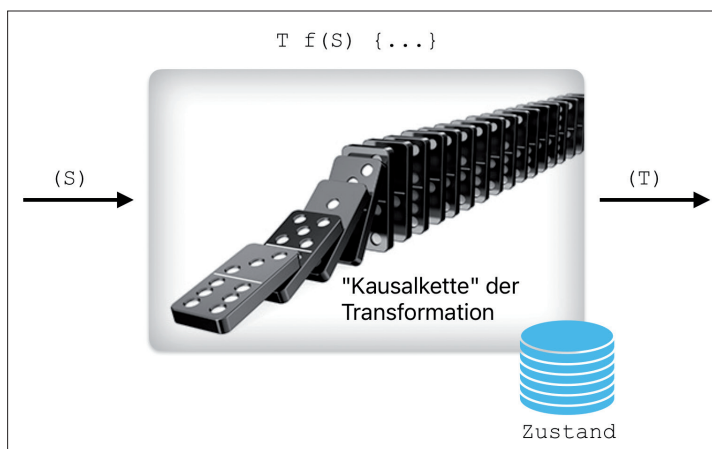
Und Verständnis ist nicht mit Lösungsansatz oder Lösung zu verwechseln. Im Zweifelsfall verstehen Sie – doch Sie erkennen noch nicht, wie eine Lösung aussehen könnte.

Bisher waren Sie derjenige, der sowohl Verständnis als auch die Lösung entwickeln musste. Mit Auftreten der großen Sprachmodelle werden wir diese allerdings zunehmend beides übernehmen lassen. Heute ist der Scope noch klein, für den das in genügender Qualität funktioniert. Doch ich bin gewiss, er wird wachsen.

Damit dieses Potenzial ausgeschöpft werden kann, bleibt Klarheit der Anforderungen wichtig. Domänenexperten inklusive POs werden es weiterhin schwer haben, ihre Wünsche präzise genug alleine zu formulieren. KI-gestützte Tools werden sich ihnen zwar anbieten, doch eine Lücke wird bestehen bleiben. Dort ist „Hebammenkunst“ gefragt, für die Sie als Entwickler prädestiniert sind.

Ausblick

Um weniger Zeit in Iterationen zu verschwenden, um dem Softwareentwurf einen besseren Ausgangspunkt



Anforderungen müssen eine Idee von der Kausalkette vermitteln, die Input zu Output transformiert (Bild 9)

zu bieten und um KI als Co-Pilot in der Programmierung einsetzen zu können, ist ein kritischer Blick auf die Anforderungsanalyse nötig. Sie als Softwareentwickler sollten einen Anspruch daran haben, was formal das Ergebnis einer Anforderungsanalyse sein soll, damit Sie Ihre eigentliche Aufgabe der Herstellung von Code am besten erfüllen können.

Für mich besteht dieser Anspruch in hoher Testbarkeit. Mit der Arbeit an Produktionscode sollten Sie nicht beginnen, bevor Sie aus dem Gespräch mit dem PO nicht das Tupel (*Funktionssignaturen, Testfälle*) für seine Anforderungen abgeleitet haben. Zusätzliche Beschreibungen einer „Kausalkette“ für die Transformation der Daten der Testfälle durch eine Lö-

Zusammenarbeit. Auch der PO wird es Ihnen danken, dass Sie ihn nicht mit Rückfragen stören.

So viel als Zielbild für die Anforderungsanalyse aus Ihrer Sicht als Softwareentwickler. Es ist überschaubar, finden Sie nicht? Mir fällt die Programmierung viel leichter, seitdem ich diesem Zielbild folge. Die Energie in Anforderungsanalysegesprächen ist eine ganz andere. Ich fühle mich nicht mehr als „Opfer“ oder „Empfänger“, sondern als Mitgestalter auf Augenhöhe mit dem PO.

Einen Haken hat das Zielbild jedoch: Sie können zu den Funktionen als Ansatzpunkte für Testfälle nicht springen. Bei realistischen Anforderungen liegen diese nicht auf der Hand.

Schon im ersten Beispiel hier haben sie Sie nicht angesprochen; vielmehr habe ich sie für Sie herausgearbeitet – ohne Ihnen zu sagen, was für mich dafür die Methode oder das Big Picture war.

Testfälle brauchen Funktionen, auf die sie angesetzt werden.

Die zur Umsetzung von Anforderungen nötigen/nützlichen Funktionen liegen nicht auf der Hand.

Die Anforderungsanalyse ist vielmehr ein eigener Ent-

wicklungsprozess mit Inkrementen und Iterationen, dessen Ziel nicht lauffähiger Code, sondern ausgefüllte Templates wie oben formuliert sind.

Auch dieser Entwicklungsprozess profitiert von Systematik. Die möchte ich Ihnen mit Slicing bieten. Damit bewegen Sie sich in klar definierten Schritten durch Anforderungen von außen nach innen, vom Groben zum Detail, vom breiten Scope zu dünn geschnittenem.

Jeder Schritt dreht sich um etwas, das für den PO greifbar ist und gleichzeitig für Sie eine konkrete Entsprechung im Code hat. Im Slicing nehmen Sie den PO an die Hand. Gemeinsam reisen Sie „in verschiedenen Zoomstufen“ durch die Anforderungen und Codestrukturen. Ein zusätzlicher Nutzen des Slicing: Es unterstützt Sie bei der Strukturierung Ihres Codes; es ist verbunden mit einem sauberen Architekturstil. Analyse und Entwurf überlappen mit dem Slicing.

Im nächsten Artikel geht es los mit einem Blick auf Anforderungen aus großer Flughöhe. ■



Ralf Westphal

ist Trainer, Berater und Mitgründer der Clean Code Developer Initiative (<https://clean-code-developer.de>). Seine Schwerpunkte sind dauerhaft hohe Produktivität für die Softwareentwicklung und zukunftsfähige Teamorganisation.

<https://ralfw.de>

dnpCode

A2404Slicing

```

greeta.sh
1  #!/bin/bash
2
3  echo "Hallo, $1!"

greetb.sh
1  #!/bin/bash
2
3  echo "Bitte geben Sie Ihren Namen ein:"
4  read name
5
6  echo "Hallo, $name!"
7

ralfw@CruelSummer temp % ./greeta.sh Balin
Hallo, Balin!
ralfw@CruelSummer temp % ./greetb.sh
Bitte geben Sie Ihren Namen ein:
Dwalin
Hallo, Dwalin!
ralfw@CruelSummer temp %

```

Prototypen für alternative
Benutzerschnittstellen
(Bild 10)

sung sind natürlich auch hilfreich – doch darüber dürfen Sie das Tupel nicht vergessen.

Sie kennen das agile Template für User Stories: „As a (role), I want (function) so that (business value)“, bestehend aus role, goal, reason oder who, what, why.

Setzen Sie dem ein eigenes Template für sich selbst als Entwickler gegenüber, mit dem Sie die Qualität von Anforderungen aus Ihrer Perspektive beurteilen: „A function with signature <signature> to satisfy the test <test case> by implementing <causal chain>.“

Wie gesagt: Es ist okay, diese formale Klarheit nicht sofort zu haben. Das ist keine Schande. Auch der PO sollte das verstehen. Es ist vielmehr bei komplexeren Anforderungen zu erwarten. Also: Keine Panik.

Die passende Reaktion ist dann entweder eine Verringerung des Scope oder ein Prototyp. Beides ist keine Zeitverschwendung. Die ist vielmehr unvermeidbar, wenn Sie zu früh, das heißt bei mangelnder Klarheit, beginnen, Produktionscode zu verändern. Dann kommt es zu unnötigen, risikoreichen Iterationen.

Für Arbeit am Produktionscode ist „full kitting“ nötig, um sie schnellstmöglich abzuschließen. Erst dann können ja die Testfälle zum Einsatz kommen für die „Reifeprüfung“. „Full kitting“ bedeutet, dass Sie sorgfältig überprüfen, ob Sie schon alles haben, was Sie für die Arbeit am Produktionscode brauchen. Ist alles zur Hand? Sind alle Fragen geklärt? Fühlen Sie sich in der Lage, die Reife selbst (mittels Tests) zu prüfen? Rückfragen sind Iterationen und verzögern Ihre Lieferung. Vermeiden Sie Rückfragen durch gute Vorbereitung in